

FACTORY FLOOR LAYOUT & MANUFACTURING OPTIMIZER

Technical System Documentation & User Manual

Edition 7.0: Perpendicular Manifold Adjacent-Offset Solver & Zero-Corridor Encroachment

SYSTEM SPECIFICATION & IMPLEMENTATION GUIDE

1. WORKSPACE LOGISTICS & SEGREGATION PHILOSOPHY

On modern industrial shop floors, material transit lanes and manned corridors are strictly protected assets. Machinery and operator clearances must never encroach upon active transit corridors, as blockages can lead to critical logistical delays, collisions, and workplace safety standard violations.

To enforce this segregation strictly, **Edition 7.0** introduces a **Perpendicular Manifold Adjacent-Offset Solver**. Instead of placing machinery centers directly on the material flow polyline path (which blocks material movement), this upgraded layout engine shifts machinery completely adjacent to the path. Equipment envelopes and required operator clearance zones sit perfectly tangent to the forklift corridor boundaries, keeping the corridor **100% free of obstruction**.

Core Improvements in Edition 7.0:

- **Strict Corridor Protection:** The part flow polyline and its surrounding forklift corridor (half-width defined by the path_buffer slider) are treated as a strictly protected exclusion zone.
- **Perpendicular Tangency Solver:** Center coordinates are shifted perpendicularly away from path segments. Spacing is dynamically computed so that each machine's outer safety standoff envelope sits perfectly tangent to the forklift corridor boundary.
- **Unified 1D Parametric Packing:** Optimization operates on linear distances s along the polyline path while coordinates (X, Y) are mapped using perpendicular normal offsets.

2. MATHEMATICAL FORMULATIONS

2.1 Dynamic Perpendicular Tangency Shifting

For any cumulative distance s along the dynamic 5-point polyline path, the system calculates the raw 2D path coordinates (X_{path}, Y_{path}) and determines the unit direction vector $\mathbf{u} = (u_x, u_y)$ of the active segment.

The perpendicular unit normal vector $\mathbf{n}$ pointing to the left of the path vector is:

$$\mathbf{n} = (-u_y, u_x)$$

To keep the machine footprint and its required standoff boundary out of the transit corridor, the machine center (X_i, Y_i) is shifted along the normal vector by a custom-calculated offset distance:

$$X_i = X_{path} + \text{Offset}_i \cdot n_x, \quad Y_i = Y_{path} + \text{Offset}_i \cdot n_y$$

The offset distance (Offset_i) is calculated dynamically per machine to make its outer safety envelope perfectly tangent to the forklift corridor boundary line:

$$\text{Offset}_i = \text{path_buffer} + (\text{dim_y}_i / 2.0) + \text{so_ny}_i$$

- **path_buffer:** Half-width of the forklift safety corridor.
- **dim_y_i:** Physical height of Machine i .
- **so_ny_i:** Negative Y Standoff (clearance buffer on the side of the machine facing the conveyor corridor).

This equation guarantees that no matter what angle a polyline segment lies on (vertical, horizontal, or diagonal), the machine's physical footprint and its standoff envelope sit completely outside the forklift lanes, ensuring zero encroachment.

2.2 Unified Overlap & Out-of-Bounds Solvers

AABB overlap solver checks are executed on the final side-shifted safe bounding boxes during optimization:

$$\text{Overlap} = (\text{Safe Min } X_1 < \text{Safe Max } X_2) \text{ AND } (\text{Safe Max } X_1 > \text{Safe Min } X_2) \text{ AND } (\text{Safe Min } Y_1 < \text{Safe Max } Y_2) \text{ AND } (\text{Safe Max } Y_1 > \text{Safe Min } Y_2)$$

3. COMPLETED REFACTORED C BACKEND ENGINE

Save this source file as optimizer.c in your repository. It contains the polyline interpolation routines, perpendicular offset calculators, soft-penalty spacing evaluation functions, and JSON results output.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <stdbool.h>
#define MAX_MACHINES 15
#define GRID_SIZE_X 20
#define GRID_SIZE_Y 20
typedef struct {
    int id;
    char name[30];
    double s; /* Cumulative distance along polyline */
    double x; /* Calculated Cartesian X coordinate */
    double y; /* Calculated Cartesian Y coordinate */
    double dim_x; /* Geometric Width */
    double dim_y; /* Geometric Height */
    double so_px; /* Positive X Standoff */
    double so_nx; /* Negative X Standoff */
    double so_py; /* Positive Y Standoff */
    double so_ny; /* Negative Y Standoff */
    double process_time;
    double setup_time;
} Machine;
typedef struct {
    int src_id;
    int dest_id;
    double volume;
} MaterialFlow;
Machine machines[] = {
    {1, "Raw Material Intake", 0.0, 0.0, 0.0, 3.0, 2.0, 1.0, 1.0, 1.0, 1.0, 2.0, 5.0},
    {2, "CNC Milling", 10.0, 0.0, 0.0, 4.0, 4.0, 1.5, 1.5, 1.5, 1.5, 8.5, 20.0},
    {3, "Laser Welder", 20.0, 0.0, 0.0, 3.0, 3.0, 1.2, 1.2, 1.2, 1.2, 4.0, 15.0},
    {4, "Surface Treatment", 30.0, 0.0, 0.0, 5.0, 3.0, 1.0, 1.0, 1.0, 1.0, 6.0, 10.0},
    {5, "Quality Assembly", 40.0, 0.0, 0.0, 3.5, 2.5, 0.8, 0.8, 0.8, 0.8, 5.0, 8.0}
};
int num_machines = 5;
MaterialFlow flows[] = {
    {1, 2, 120.0},
    {2, 3, 100.0},
```

```

        {3, 4, 80.0},
        {4, 5, 95.0},
        {2, 5, 15.0}
    };
    int num_flows = 5;
    /* Dynamic 5-point polyline coordinates */
    double polyline_x[5] = {3.00, 3.00, 10.00, 17.00, 17.00};
    double polyline_y[5] = {3.00, 17.00, 17.00, 10.00, 3.00};
    double path_buffer_val = 1.00; /* Configurable forklift corridor half-width */
    double calculate_center_distance(double x1, double y1, double x2, double y2) {
        return sqrt((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2));
    }
    void get_offset_point_on_polyline(double s, double offset_dist, double *out_x, double *out_y) {
        double seg_lens[4];
        int i;
        for (i = 0; i < 4; i++) {
            seg_lens[i] = sqrt((polyline_x[i+1]-polyline_x[i])*(polyline_x[i+1]-polyline_x[i]) +
                               (polyline_y[i+1]-polyline_y[i])*(polyline_y[i+1]-polyline_y[i]));
        }

        double current_s = 0.0;
        for (i = 0; i < 4; i++) {
            if (s <= current_s + seg_lens[i]) {
                double ratio = (s - current_s) / seg_lens[i];
                double px = polyline_x[i] + ratio * (polyline_x[i+1] - polyline_x[i]);
                double py = polyline_y[i] + ratio * (polyline_y[i+1] - polyline_y[i]);

                double dx = polyline_x[i+1] - polyline_x[i];
                double dy = polyline_y[i+1] - polyline_y[i];
                double ux = dx / seg_lens[i];
                double uy = dy / seg_lens[i];

                double nx = -uy;
                double ny = ux;

                *out_x = px + offset_dist * nx;
                *out_y = py + offset_dist * ny;
                return;
            }
            current_s += seg_lens[i];
        }
        *out_x = polyline_x[4];
        *out_y = polyline_y[4];
    }
    double get_machine_offset(Machine m, double path_buffer) {
        return path_buffer + (m.dim_y / 2.0) + m.so_ny;
    }
    bool check_safe_overlap(double x1, double y1, double w1, double h1, double px1, double nx1, double
py1, double ny1,
                           double x2, double y2, double w2, double h2, double px2, double nx2, double
py2, double ny2) {
        double min_x1 = x1 - w1 / 2.0 - nx1;

```

```

double max_x1 = x1 + w1 / 2.0 + px1;
double min_y1 = y1 - h1 / 2.0 - ny1;
double max_y1 = y1 + h1 / 2.0 + py1;

double min_x2 = x2 - w2 / 2.0 - nx2;
double max_x2 = x2 + w2 / 2.0 + px2;
double min_y2 = y2 - h2 / 2.0 - ny2;
double max_y2 = y2 + h2 / 2.0 + py2;

return min_x1 < max_x2 && max_x1 > min_x2 && min_y1 < max_y2 && max_y1 > min_y2;
}

bool is_safe_out_of_bounds(double x, double y, double w, double h, double px, double nx, double py,
double ny) {
    return (x - w/2.0 - nx < 0.0) || (x + w/2.0 + px > GRID_SIZE_X) || (y - h/2.0 - ny < 0.0) || (y +
h/2.0 + py > GRID_SIZE_Y);
}

double evaluate_polyline_layout(void) {
    double transport_cost = 0.0;
    double penalty = 0.0;
    int i, j;
    for (i = 0; i < num_machines; i++) {
        double offset_dist = get_machine_offset(machines[i], path_buffer_val);
        get_offset_point_on_polyline(machines[i].s, offset_dist, &machines[i].x, &machines[i].y);
    }
    for (i = 0; i < num_flows; i++) {
        int src = flows[i].src_id;
        int dest = flows[i].dest_id;
        int idx_src = -1, idx_dest = -1;
        for (j = 0; j < num_machines; j++) {
            if (machines[j].id == src) idx_src = j;
            if (machines[j].id == dest) idx_dest = j;
        }
        if (idx_src != -1 && idx_dest != -1) {
            double dist = calculate_center_distance(machines[idx_src].x, machines[idx_src].y,
machines[idx_dest].x, machines[idx_dest].y);
            transport_cost += dist * flows[i].volume;
        }
    }
    for (i = 0; i < num_machines; i++) {
        if (is_safe_out_of_bounds(machines[i].x, machines[i].y, machines[i].dim_x,
machines[i].dim_y,
machines[i].so_px, machines[i].so_nx, machines[i].so_py,
machines[i].so_ny)) {
            penalty += 10000.0;
        }
        for (j = i + 1; j < num_machines; j++) {
            if (check_safe_overlap(machines[i].x, machines[i].y, machines[i].dim_x,
machines[i].dim_y,
machines[i].so_px, machines[i].so_nx, machines[i].so_py,
machines[i].so_ny,
machines[j].x, machines[j].y, machines[j].dim_x,
machines[j].dim_y,

```

```

                                machines[j].so_px, machines[j].so_nx, machines[j].so_py,
machines[j].so_ny)) {
    penalty += 20000.0;
}
}
}
return transport_cost + penalty;
}

```

4. FRONTEND STREAMLIT DESIGN & VISUAL MARKINGS

The Streamlit interface enables interactive, adjacent-offset routing configurations.

4.1 Protected Corridor & Adjacent-Offset Visualization

The Matplotlib plotting utility maps the protected forklift corridor and shifts machine footprint rectangles completely adjacent to its boundaries:

- **Conveyor Backbone:** Plotted as a solid, high-visibility grey path line passing sequentially through the dynamic vertices.
- **Forklift safety lane boundary:** Drawn as parallel yellow dashed lines offset at $\pm$ path_buffer from the polyline path, denoting the protected corridor.
- **Perpendicular Offset Machine Footprints:** Solid rectangles representing physical equipment footprints are rendered perpendicularly offset to the left of the conveyor line. The bottom edge of each machine's physical envelope sits perfectly tangent to the forklift corridor boundary line.
- **Perpendicular Offset Safety Envelopes:** Dashed envelopes representing standoff limits are drawn offset from the machine edges, keeping the forklift lane free of any physical or buffer encroachment.

5. RECTANGULAR LAYOUT USER MANUAL

Launch the application to build and optimize adjacent-offset layouts.

5.1 Step-by-Step Configuration Flow

1. **Commit Code Updates:** Push the latest app.py and optimizer.c source code files to your GitHub repository.
2. **Deploy on Streamlit Community Cloud:** Launch the dashboard in your web browser.
3. **Define Forklift Corridor Width:** Use the sidebar slider **Fork Truck Safety Corridor** to specify your forklift clearance width (e.g., set to 1.2m).
4. **Edit Path Vertices:**
 - Navigate to **Tab 1: Configure Parameters**.
 - Edit the **Dynamic Vertices** table coordinates to define your conveyor path.
5. **Edit Machine Parameters:**
 - In the **Machine Spacings** table, configure spacings (s), footprints, and standoff clearances.
 - Ensure the standoff buffer on the side facing the conveyor (the Negative Y standoff so_ny) is configured

correctly.

6. Run Optimization Heuristics:

- Navigate to **Tab 3: Execution & Visual Analytics** and click **Run Layout Optimizer Heuristic**.
- The engine evaluates cumulative distances along the custom path segments, packing machines together to minimize transport costs while ensuring safety zones do not overlap.

7. Interpret Outputs:

- Confirm that your machine footprints and safety boundaries are placed completely adjacent to the yellow dashed forklift corridor boundary.
- Verify that the forklift safety lane remains completely unobstructed.

6. DESIGN FAQ & TROUBLESHOOTING

6.1 Bounding Box Protrusion Errors

- **Symptom:** A machine is flagged with warning errors or sits outside the room boundaries after optimization.
- **Explanation:** Shifting the machines perpendicularly away from the polyline path pushes their outer borders past the grid walls.
- **Resolution:** Increase the floor width (X) or height (Y) using the sidebar sliders, or move the path vertices closer to the center of the room.

6.2 Conveyor Adjacency Verification

- **Symptom:** Machines are spaced too far away from the conveyor line.
- **Explanation:** The required perpendicular offset is calculated dynamically per machine to make its outer safety envelope perfectly tangent to the forklift corridor boundary line. If the standoff buffer `so_ny` is set too high, the machine will sit unnecessarily far from the conveyor.
- **Resolution:** Reduce the Negative Y standoff (`so_ny`) parameter value in Tab 1, then re-execute the optimizer.